The letter as a program: constructing the Chuzhditsa typeface

Author name withheld for review*

Abstract

This is a case study in parametric typeface construction for a designed Cyrillic orthographic register. Chuzhditsa is a four-style Cyrillic family (Regular, Bold, Italic, Bold Italic; TTF and CFF) generated entirely from centerline primitives — lines, arcs, rings — by a stroke-expansion engine in which weight, slant, terminals, joins, fitting, and a canon of optical corrections are code. We document the engine and, in detail, the corrections that separate a naïvely expanded stroke font from a typographically credible one: butt and cut terminals, mitered path chaining, curvature-clamped weight, widened bold advances, overshoots and optical centering. We then show how the font's OpenType layer functions as an executable reference implementation of the orthography it serves — ligature lookup order, computed mark anchors, and normalization coverage make prose-level spelling rules machine-testable — and close with a small instrumented evaluation of mark survival and confusable-pair contrast at reading sizes. The conceptual claim is deliberately scoped: parametric type fails as a theory of all letterforms and succeeds as a theory of one design language at a time.

1 Introduction: the oldest new idea

Every typeface is a theory of writing frozen at some level of abstraction. A drawn font freezes finished shapes; a hinted font freezes shapes plus advice to the rasterizer; Chuzhditsa freezes something earlier in the pipeline — the *gesture*: each glyph is stored as centerline strokes annotated with a construction (ring, arc, chained line), and everything visible about the letter (weight, slant, terminal form, joint behavior, optical compensation) is computed at build time by roughly seven hundred lines of Python.

Type has migrated through abstraction levels before, and each migration changed what a "design" is. The punchcutter's design was steel, one size at a time, and the trade's late self-description runs to seven hundred pages of machines (Legros & Grant 1916); Benton's pantograph made the drawing the design and the sizes derivatives (cf. Southall 1985 on pattern drawings and the designer–producer split); Ikarus made the digital outline the design and the drawing a preliminary (Karow 1994); OpenType made the behavior — substitution, positioning — part of the design; variable fonts made the interpolation space itself the design. Chuzhditsa takes the next step back along the pipeline and makes the *construction* the design (Figure 1).

The idea is old. Knuth proposed in 1982 that a typeface could be a program whose parameters — weight, slant, contrast — generate a family from one description (Knuth 1982, 1986). Hofstadter's celebrated reply argued that the space of all 'A's is not parameterizable: letterforms are an open category, and no finite knob-set spans them (Hofstadter 1982). Practice sided with Hofstadter at the ambitious end of the claim while quietly vindicating the modest end — parametric and interpolated workflows are now routine within scoped design spaces; Southall's blunt finding from the sympathetic side was that designers' craft knowledge "is not usually available to them in a form in which it can be articulated

*Companion paper to *Chuzhditsa: a featural citation register for Cyrillic*; the build toolchain discussed here is roughly seven hundred lines of Python and is distributed with the fonts. Every object-language example in this manuscript is set in the typeface under discussion.

symbolically" — it lives in shapes, not statements — and that METAFONT was therefore "unbelievably difficult to use as a tool for making new typeface designs" (Southall 1985). But the argument was about the wrong quantifier. Knuth's dream fails for *all* letterforms and succeeds for *one design language at a time*: within a single monoline geometric grammar, weight and slant genuinely are parameters, and the Bold and Italic of Chuzhditsa are not drawings but evaluations. This is also, we note, the position the industry itself eventually reached by another road — variable fonts interpolate within one design space and claim nothing beyond it.

The contemporary landscape should be acknowledged before the historical one is mined. Single-stroke lettering has its own lineage from Hershey's plotter fonts (Hershey 1967) through engraving and CNC single-line faces; production type design today runs on scriptable editors (RoboFont, Glyphs) whose Python layers routinely generate, correct, and test glyphs; designspace-based variable-font workflows treat a family as an interpolation region; Metapolator and its kin attempted user-facing parametric design; and skeleton-based generation is an active strategy for very large CJK character sets. Chuzhditsa differs from these mainly in degree of commitment: there is no drawn master anywhere in the pipeline — the skeleton declarations and the engine are the only sources — and the OpenType behavior is generated from the same specification as the shapes.

Five terms do load-bearing work in what follows and are worth fixing now. The *orthography* is the mapping from phonological input to character sequences; the *register* is the functional category being marked (foreign-word citation); the *typeface* is the design language; the *font binary* is the executable artifact — glyphs, cmap, GSUB/GPOS, anchors; and the *text encoding* is the Unicode character sequence, which preserves the orthography's letters in plain text but preserves the register only where font control survives.

There was also a practical reason to build rather than draw. The companion paper describes Chuzhditsa's function: a *phonological citation register* for pan-Slavic Cyrillic — a system for writing foreign words, names, and pedagogical examples so a Cyrillic reader can recover the source pronunciation, with loanword marking as a secondary use — whose extension letters are derived by a small set of recurrent diacritic operations, some featurally atomic and some relational to the base letter. A writing system whose *letters* are rule-derived invites a typeface whose *glyphs* are rule-derived; the two grammars mirror each other, and several orthographic rules (ligature fusion, mark anchoring) exist in the font as executable code rather than prose. The font is not an illustration of the spec; it functions as its *executable reference implementation* — the artifact against which claims about fusion, attachment, and coverage can be tested mechanically (§7). The construction method is evaluated on its own terms throughout: nothing below depends on the reader's verdict on the register's adoption prospects. Given a designed orthographic register — this one or another — the question this paper answers is what it takes to make it typographically real.

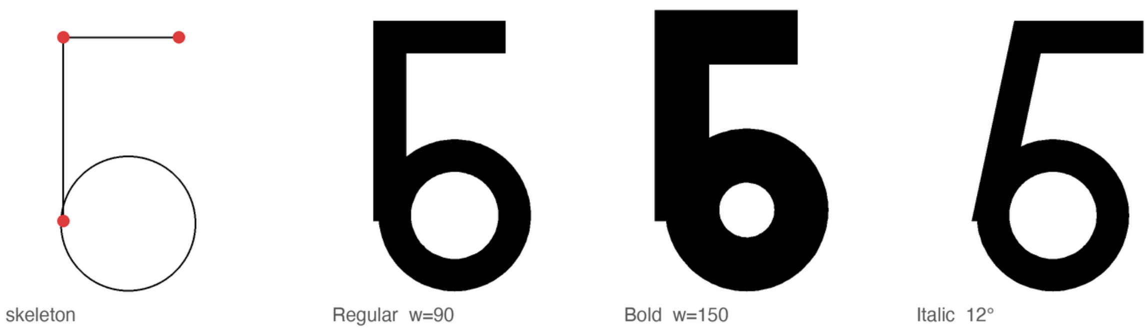

Figure 1: The stroke model. Left: the declared skeleton of б — two line strokes (endpoints marked) and one ring. Right: the same declaration evaluated at $w = 90$, $w = 150$, and 12° slant. Nothing between the panels was drawn.

2 The stroke model

Noordzij's theory holds that the letter is built of strokes, each the trace of a frontline segment — his counterpoint — advancing along a path, with the distribution of weight about the path defining the style (Noordzij 2005). Chuzhditsa is that theory made executable in its simplest case. Every glyph is declared as centerline primitives — straight segments, circular arcs, full rings, dots — in a 1000-unit em, capitals 700 high, lowercase 500. The expansion engine sweeps a disc of diameter w along each path: $w = 90$ units yields Regular, $w = 150$ Bold. The italic applies a 12° shear *to the centerlines before expansion*, not to the finished outlines; a sheared outline modulates its own thickness (the singular values of a 12° shear are about 0.90 and 1.11, a visible ±10% wobble around a ring), whereas a sheared centerline swept by an unsheared disc keeps constant perpendicular weight — which recovers one specific behavior of pen construction — constant stroke thickness normal to the path — and is why the pen tradition (Johnston 1906) remains the right mental model even for a face with none of the pen's contrast (Figure 2).

The entire artwork of a letter is a line of Python. Here is b, in full:

```
"b": dict(adv=660, strokes=[(R,340,190,180),      # ring: bowl
                            (L,160,190,160,700),  # line: stem
                            (L,160,700,480,700)]) # line: top arm
```

Everything else the reader sees in Figure 1 — the weight, the crisp corner where stem meets arm, the open counter in the Bold, the slope of the Italic — is the engine's doing, not the declaration's. The primitive vocabulary is deliberately small — line, arc, ring, dot — because every primitive is a maintenance contract. A ring is not "a circle the designer drew" but a promise that bowls will respond as a class to any future correction: when Bold needed open counters, one clamp (stroke may not exceed $0.85r$) repaired every bowl in the family at once; when the big rounds needed optical narrowing, one factor did the same. This is the practical content of the parametric claim, and it is worth stating against the romance of the drawn glyph: in a 26-letter extension set serving thirty-plus orthographies, corrections that do not generalize do not get made. Bigelow and Holmes reported the same economics from the first large Unicode face: coherence across thousands of glyphs is the product of harmonization — "basic weights and alignments…regularized and tuned to work together" — because glyph-by-glyph invention does not fit inside a designer's working life (Bigelow & Holmes 1993).

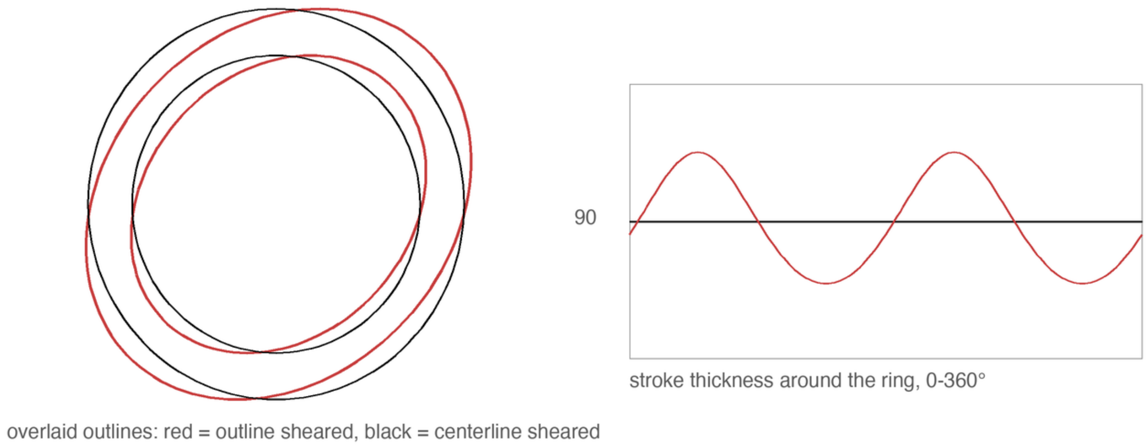

Figure 2: Two ways to make an italic O. Left: shearing the finished outline modulates the ring's own thickness. Right: shearing the centerline and sweeping an unsheared disc preserves constant perpendicular weight — the pen's behavior, recovered arithmetically.

Monoline is the least-ink case of Noordzij's model and it is easy to mistake for the easy case. It is the opposite. Contrast hides sins: where a modulated face can thin a stroke through a crowded joint, a monoline must resolve every collision honestly, in geometry. Most of what follows is the record of those resolutions.

3 Why it looks the way it looks: Cyrillic, Glagolitic, and a register face

3.1 Cyrillic skeleton constraints

The visual register must be recognizably *other* at reading distance while remaining effortlessly legible to any Cyrillic reader — the katakana condition. The letterforms therefore keep utterly conventional Cyrillic skeletons (Zhukov's account of what makes Cyrillic Cyrillic — the correlated treatment of А Л Д М, of Б В Р Ч Ъ Ы Ь Я, of Ж with К and Я, and the dominance of verticals and straight-line construction in the lower case — was treated as a constraint list; Zhukov 1996), while the *style* carries the register.

3.2 Glagolitic register styling

The style's sources are deliberately archaic: the full-circle bowls of round Glagolitic, the first Slavic script, whose loops survive in our б в р ъ ь ы ю; and a unicameral *memory*: the family is fully two-case in function (§6), but capitals and lowercase share one skeleton grammar and one construction vocabulary, echoing pre-Petrine Cyrillic before the civil type reform imported Latin two-case architecture — a stylistic inheritance, not a functional restriction. For a Bulgarian-born project there is a further conversation to honor: Bulgarian typography has long maintained its own letterform norm distinct from the Russian model (the tradition whose modern theory Vasil Yonchev shaped), and a face that revives ѫ and ѩ is, in that discourse, taking a side — heritage forms for heritage letters (Йончев & Йончева 1982).

3.3 Pan-Slavic conflicts: the е/є case

Working at the scale of the whole Slavic superset also disciplines individual letters in ways single-language design never would. The pilot's most charming glyph — a wide, round е, the historic "broad e" — turned out to be sitting on Ukrainian's chair: the shape *is* є (U+0404), a distinct letter with its own phonemic identity. Pan-Slavic coverage forced е to an angular form so that the е/є contrast survives, a correction invisible to a Bulgarian reader and load-bearing for a Ukrainian one. The general rule we draw: in a multi-orthography script, the letterform space is commons, and a design that grazes it for one language will silently fence off a neighbor's letter. Design against the superset first.

4 What separates drawn-looking from designed: four failures and their remedies

The first complete build of Chuzhditsa was, in the frank words of its commissioning review, "handwritten by a child." The diagnosis decomposed into four specific engineering failures, each with a specific remedy; we document them because the literature is rich in descriptions of good type and poor in descriptions of the exact distance between good and bad — and because none of the four is specific to Chuzhditsa: each is a general failure mode of centerline-expanded monoline type, and each remedy transfers.

4.1 Terminals: the marker-pen effect

The naive stroke sweep caps every terminal with a semicircle — the mark of a felt-tip, not a punch. The remedy is the butt cap, perpendicular to the stroke; and for straight diagonals meeting the baseline or cap line, a further convention from the drawing office: the terminal is cut *horizontally*, so that Л А Д stand on flat feet rather than teetering on angled ones (Figure 3). Curves keep perpendicular terminals (c ϵ э) — a horizontal cut mid-arc would be a wound, not a foot. The discrimination is done by the engine, not the designer: a terminal is cut flat when its stroke is straight, longer than a threshold, and steeper than $\sim$19° — three predicates that encode, crudely but adequately, the difference between a stem and a serifless flourish.

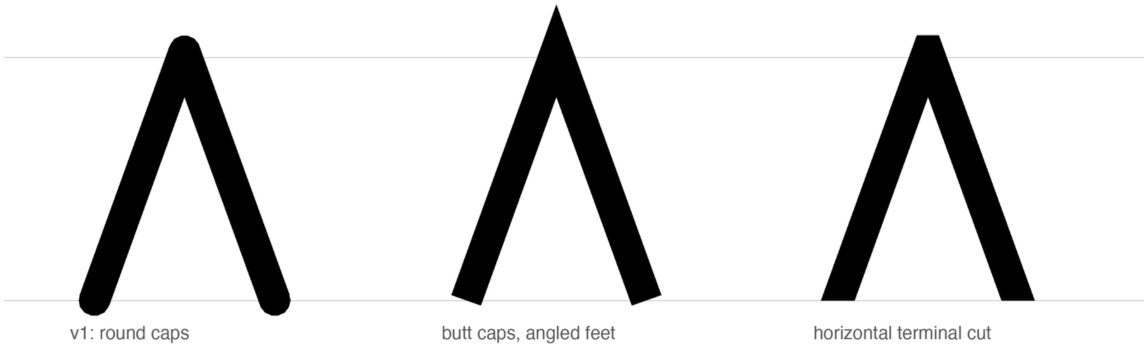

Figure 3: Three terminal policies on Λ. Round caps read as marker pen; perpendicular butt caps stand the letter on angled feet; the horizontal cut plants it. The apex keeps its miter throughout — a point may pierce the cap line; a letter should not balance on one.

4.2 Joints: the punchcutter's corner

Strokes that merely overlap produce lumps where a corner should be. The engine therefore *chains*: strokes sharing an endpoint (where exactly two meet) are merged into one path and their meeting rendered as a mitered corner — sharp on the convex side, exactly intersected on the concave side. This is the move that turned И Λ M from taped-together sticks into letters; it is also where a subtle lesson about rasterizers was learned. A bevel on the *concave* side of a joint folds the outline over itself; every nonzero-winding rasterizer (FreeType, CoreText, every browser) renders the fold invisibly, but even-odd consumers render it as a white slit. The failure was invisible in the shipping renderer and glaring in a diagnostic one — the type-engineering equivalent of a bug that only reproduces under the debugger, and a reminder that Smeijers's dictum about counters (the white is the design; Smeijers 1996) extends to the white you create by accident (Figure 5).

The join arithmetic deserves to be written down, because it is where every stroke-model implementation we know of has stumbled at least once. At an interior vertex the two segments' offset lines are intersected; on the *convex* side the intersection (the miter) is accepted up to a limit of three stroke-widths, past which the corner is beveled — Futura-sharp apexes below the limit, honest flats beyond it. On the *concave* side the intersection must be accepted *unconditionally*: it always exists and always lies near the corner, and the tempting shortcut of beveling both sides folds the outline over itself (Figure 4). The fold is the treacherous kind of wrong: nonzero-winding rasterizers cancel it silently, and only an even-odd consumer — a naive SVG export, a diagnostic renderer — exposes the white slit that was there all along.

4.3 Curves: the facet tax

Outlines here are dense polygons rather than Béziers, which is harmless — at 96–128 samples a ring is optically indistinguishable from a true circle — until it isn't: early builds sampled arcs at 8° and the facets were visible at display sizes. Sampling is now proportional to radius. The general moral is the one Bigelow and Holmes drew from the first large Unicode face: at character-set scale, quality lives in systematic policy — harmonized weights and alignments across whole scripts — not in heroic per-glyph exceptions (Bigelow & Holmes 1993).

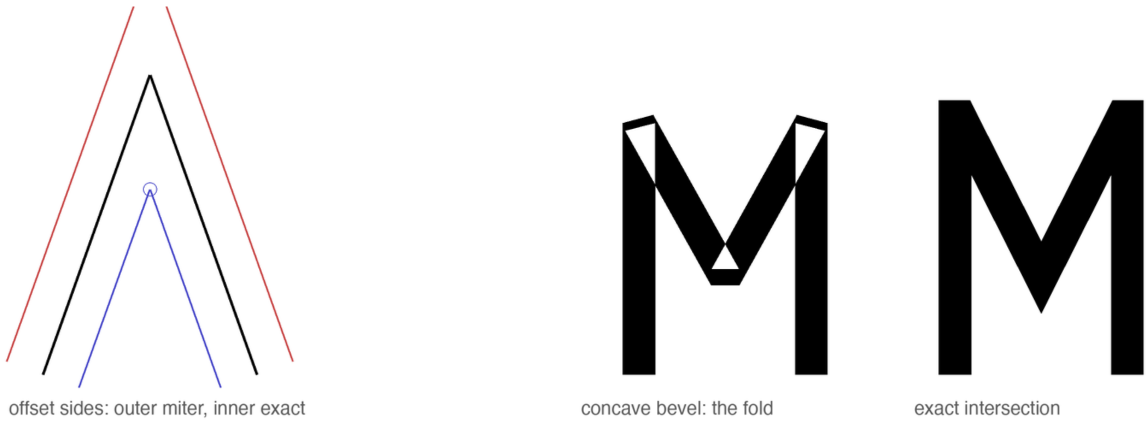

Figure 4: Join arithmetic. Left: centerline with both offset sides; the outer intersection is the miter, the inner one is mandatory. Center: bevel wrongly applied to the concave side folds the outline — under even-odd filling, `M` opens white seams. Right: the exact concave intersection.

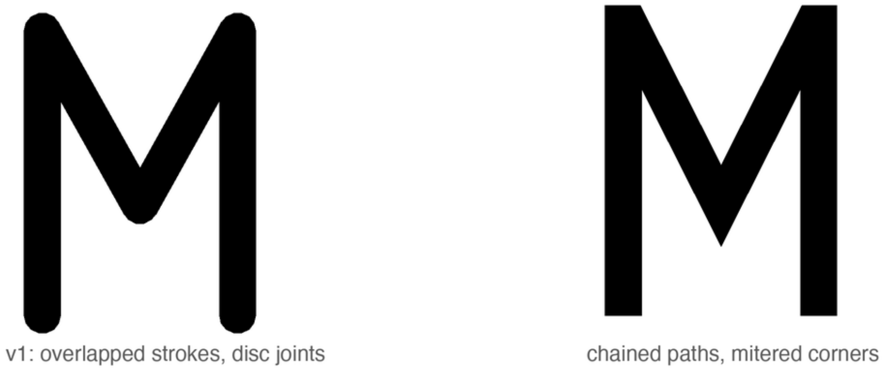

Figure 5: `M` before and after path chaining. Left: four independent strokes, their meetings papered over with discs. Right: one chained path; apex and valley are mitered intersections. The letter stops being an assembly and becomes a drawing.

4.4 Color: bold is not thick regular

A Bold whose advances do not widen is a Regular that gained weight and kept its trousers: with stroke 150 in slots spaced for stroke 90, adjacent bold `o o` overlapped outright. The Bold therefore widens every advance by the weight delta and re-centers each glyph; counters are protected by clamping stroke weight against curvature — a closed bowl of radius r carries at most $0.85r$ of stroke, an open arc at most r (an open arc always keeps half its radius of daylight, so it can afford the extra weight) — which is the programmatic form of the punchcutter's counter-first discipline (Smeijers 1996). The clamp earns its keep at the small end of the curvature range: a Bold breve is an arc of radius 130 units asked to carry a 150-unit stroke — more ink than circle — and without the clamp it ships as a blob, as do the bowls of `s` and `B` (Figure 6). The stroke narrows to what the curve can hold; nobody notices, which is the intended outcome. The result can be stated as a rule of thumb the authors have not seen printed: *in a parametric monoline, weight is a property of the page color, but advance width is a property of the counter* (Figure 7).

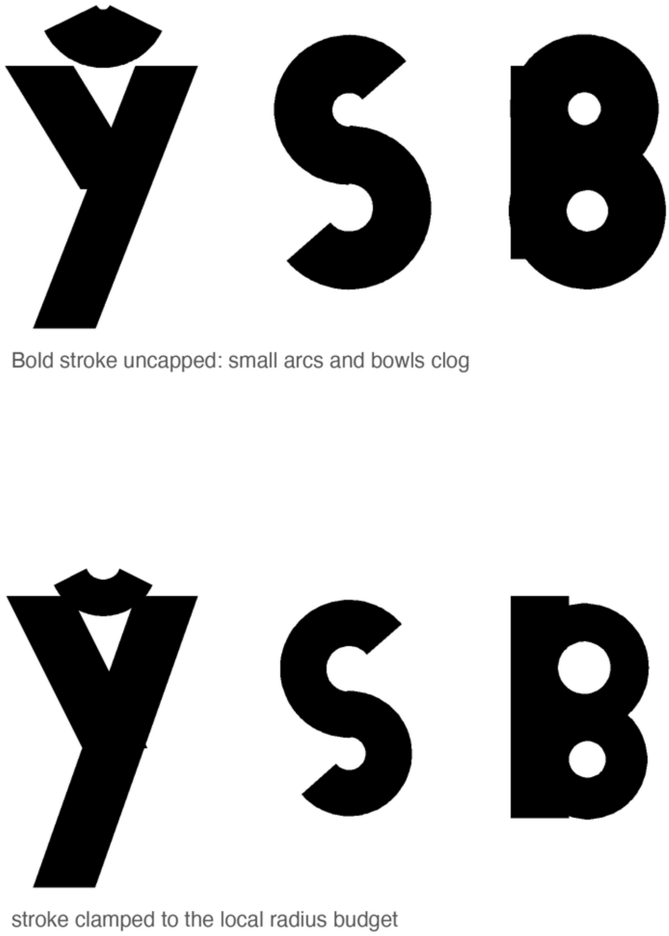

Figure 6: Curvature-clamped weight. Above: Bold's full 150-unit stroke applied to the breve of y̆ and the bowls of S B clogs them shut. Below: stroke clamped to the local radius budget ($0.85r$ for closed bowls, r for open arcs) — the counters breathe, the weight reads unchanged.

5 The optical canon, computed

Type design's oldest open secret is that geometry lies to the eye, and that every "geometric" face is a catalogue of concealed corrections — Renner's Futura most famously: its first, more strictly geometric trial forms were revised through development — the released *p* corrects an apparent stem overshoot and its thick curve-to-stem junctions — and Burke's verdict on the result is "a subtle mastery of …optical compensation" (Burke 1998: 101–102). The standard catalogue of such corrections — narrowed rounds, pierced apexes, thinned joints — is visible in any specimen. Dwiggins put the practice into designers' folklore: the "M-formula" of his 1937 memoranda to Griffith — flat planes cut so the eye reads curves (Dwiggins 1937) — and the broader counsel of *WAD to RR*: build the letter the eye wants, not the instrument (Dwiggins 1940); Tracy systematized the spacing half of the canon (Tracy 1986); the legibility literature supplies the floor beneath it all (Tinker 1963). Chuzhditsa implements the canon as build-time arithmetic:

- **Overshoot of rounds.** Flat letters read taller than round ones of equal height, so full-height rings overshoot the cap band by $\sim$10 units and undershoot the baseline equally. In a stroke model this falls out naturally: ring radii were chosen so the *outer* edge, not the centerline, aligns optically.
- **Overshoot of apexes.** A point that stops exactly at the cap line reads short — the eye averages the vanishing wedge. The apexes of A Λ A A A pierce the line by 15–18 units. (Their feet,

Figure 7: The same word, Bold strokes. Above: on Regular advances, the letters weld. Below: advances widened by the weight delta and glyphs re-centered — the counters, inner and outer, survive the weight.

conversely, are cut flat: a point may pierce a line it approaches; a letter should not balance on one.)

- **Optical center.** A crossbar at the geometric middle appears to sag. The bars of Е Н sit 18 units high — the optical-centre rule of the lettering classroom: Johnston teaches that the apparent centre lies “very slightly above mid-height” (Johnston 1906: 273–274).
- **The circle that is not.** A mathematically perfect circle beside rectangular letters reads wide; all full-height rounds (О С Є Э and kin) are compressed 3.5% horizontally. This correction is invisible, which is the point: as Bringhurst has it, typography exists to honor content, and its finest work is unnoticeable (Bringhurst 2004: 17).

Figure 8 shows all four corrections against ruled lines; each pair differs by one number in the source.

None of these numbers is sacred, and none is proposed as a universal constant: all were set by designer judgment under proofing — enlarged, squinted at, revised — and are published as the reproducible record of one face’s optical parameters, not as law. What deserves emphasis is how *small* the winning corrections are. The apex overshoot is 2.5% of the cap height; the round compression is 3.5% of a width; the bar rise is under 3%. At reading sizes not one of them is consciously visible, and together they are the entire difference between the first proof and the last. The eye audits at a precision it does not report. This is also why empirical legibility work (Tinker 1963) so rarely resolves such differences: its instruments measure reading, and these corrections operate below reading, in the layer where a face merely feels certain or feels slightly wrong.

For the record — and because a parametric face can state its entire design in one table where a drawn face cannot — Table 1 is the complete parameter sheet.

What the parametric build changes is not the eye’s authority but the cost of obeying it: a correction, once found, is a line of arithmetic applied uniformly to every present and future glyph — including the twenty-six extension letters that are this face’s reason to exist.

Parameter	Value	Parameter	Value
em / cap / x-height	1000 / 700 / 500	apex overshoot	+15–18
ascender / descender	1000 / -350	round overshoot	$\sim$10 beyond stem band
stroke: Regular / Bold	90 / 150	optical bar rise	+18
italic slant	12° (centerline)	round compression	96.5% (radius $\geq$ 200)
dot radius: Reg / Bold	50 / 66	weight clamp	ring $0.85r$; open arc r
lowercase scale (x, y)	0.72, 0.714	miter limit	3.0 stroke-widths
Bold advance widening	+60	terminal cut	straight, long, $>$19° steep
kern classes	$-15 \ldots -70$	mark anchors	bbox top $+40$; bottom floor -60
curve sampling	$\min(128, \max(48, r))$ pts	arc sampling	one point per 2.5°

Table 1: The complete parameter sheet. Every visual property of the family derives from this table plus the per-glyph skeleton declarations.

6 Fitting: space as material

Tracy's dictum that the fitting of letters is "fundamental to the success of a type design" (Tracy 1986: 71) survives translation into a parametric setting with interest: in this project, roughly half of the perceived quality gains came from decisions about white. The face carries a coarse spacing system of the classical kind — straight-sided letters hold larger sidebearings, round and open forms sit tighter, with the rounds' overshoot supplying part of their fitting — and a deliberately shallow class-kerning layer — eight glyph classes and nineteen class rules, compiled as an OpenType `kern` feature — flag-topped letters (г т ғ) against open diagonals (а л д), diagonals and у-forms against the rounds and each other, letters into low punctuation, with values from -15 to -70. The featural design pays a dividend here: extension letters join their base letters' kern classes by construction — ғ and ґ kern as г, ў as у, ӓ as а — so the extension inventory inherits fitting for free, and the worst-case pairs remain the classical ones (flag over diagonal). Kerning was kept shallow on principle: deep pair lists are where parametric discipline goes to die, since every pair is a hand-set exception to the system. Where a class rule could not fix a combination, we treated the collision as evidence about the *glyphs* — usually a sidebearing wrong on one side — and repaired the letter, not the pair. Figure 9 shows the mechanics: advance boxes with their shaded sidebearings — straights carrying about 115 units of air, rounds only 46 plus their overshoot, open diagonals in between — and the one kern class a reader will most often exercise, flag-topped letters over open diagonals. The single largest fitting decision has already been described: Bold's advances widen with its weight, which is a spacing rule wearing a weight rule's clothing. Figures and punctuation live inside the same system and the same vocabulary — the zero is a ring, the guillemets are chained diagonals, the question mark is an arc with a dot (Figure 10).

The two-case architecture is likewise a spacing story as much as a shape story (Figure 11). Lowercase is derived from the capital skeletons at x-height $500/700$, with descenders retained at full depth and marks re-seated; four letters — а б р ф — refuse derivation and carry custom constructions, because their lowercase identities (bowl-and-stem, ascender, descender) are lexical facts of Cyrillic, not scalings. The 0.72 horizontal factor was chosen so that derived lowercase keeps the capitals' stroke weight without reading condensed: scale a skeleton, never a stroke.

7 The font as executable reference implementation

"The font is the spec" would be rhetorically satisfying and technically indefensible — two fonts can implement one orthography differently, and a claim of normativity must say what conformance means. The defensible claim is narrower and more useful: the binaries are an *executable reference implementation*. The normative objects are the published phoneme-to-grapheme mapping and the shaping test suite; a font conforms if it passes the tests; this font is the reference because it is generated from the same source as the specification and ships with the tests it passes. Plain text remains the portability

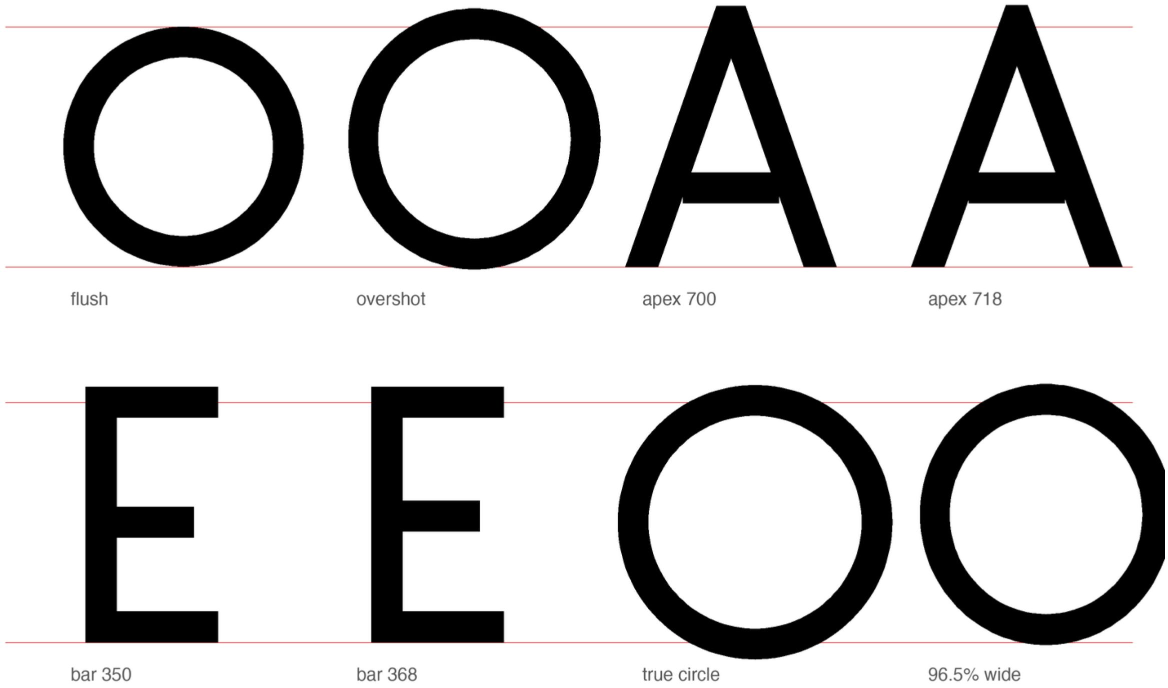

Figure 8: The optical canon, one variable at a time, on ruled cap and base lines. Flush versus overshot rounds; apex at the cap line versus pierced through it; crossbar at the geometric versus optical center; a true circle versus the shipped 96.5% width.

floor — the orthography's letters survive any font substitution; only the register's visual marking does not. Three properties give the reference implementation its teeth. First, *ligature law*: the orthography's fusions (нь and ль into their Vuk-style fused forms; ь+ѧ into the medieval ѩ; the ejective digraphs' palochka rising to half height) are GSUB rules compiled into the font, with lookup order fixed so that yus fusion outranks soft-sign fusion (Figure 13). The ordering is not pedantry: in a single unordered lookup, the rule fusing л+ь fires first on the sequence л–ь–ѧ, consumes the soft sign, and leaves the iotated-yus rule nothing to match — the documented spelling лѩ becomes unreachable, silently (Figure 12). The bug was caught by shaping tests, not by eye, because the wrong output is not ugly, merely wrong — the most dangerous species of typographic error. Concretely, the rules and their tests are plain enough to print. The fusion law is generated feature source:

```
lookup IOTYUS   { sub ermal bigyus by iotbyus; ... } IOTYUS;   # must precede
lookup SOFTFUSE { sub l ermal by lsoft; ... }        SOFTFUSE;
```

An anchor rule is one generated line per base — `pos base a <anchor 330 758> mark @TOP <anchor 330 -10> mark @BOT;` — with coordinates computed from the glyph's own bounding box. And a shaping assertion is a string paired with an expected glyph sequence, executed through HarfBuzz: `shape("лѩ") == [lc.l, lc.iotbyus]`; the normalization-closure test NFC-normalizes every spelling the orthography generates and asserts that no codepoint falls outside the cmap. Pass condition in every case: exact sequence match.

A quieter class of engineering hides in the character map. Unicode normalization composes е+grave into the precomposed ѐ (U+0450) and и+grave into ѝ (U+045D); a font that lacks those codepoints watches browsers silently substitute a fallback face for exactly those letters, defeating the register that is the whole point. The cmap therefore covers the NFC closure of every sequence the orthography can produce, and the palochka ligatures accept the caseless Ӏ (U+04C0) beside lowercase letters, because that is how Caucasian orthographies actually type it.

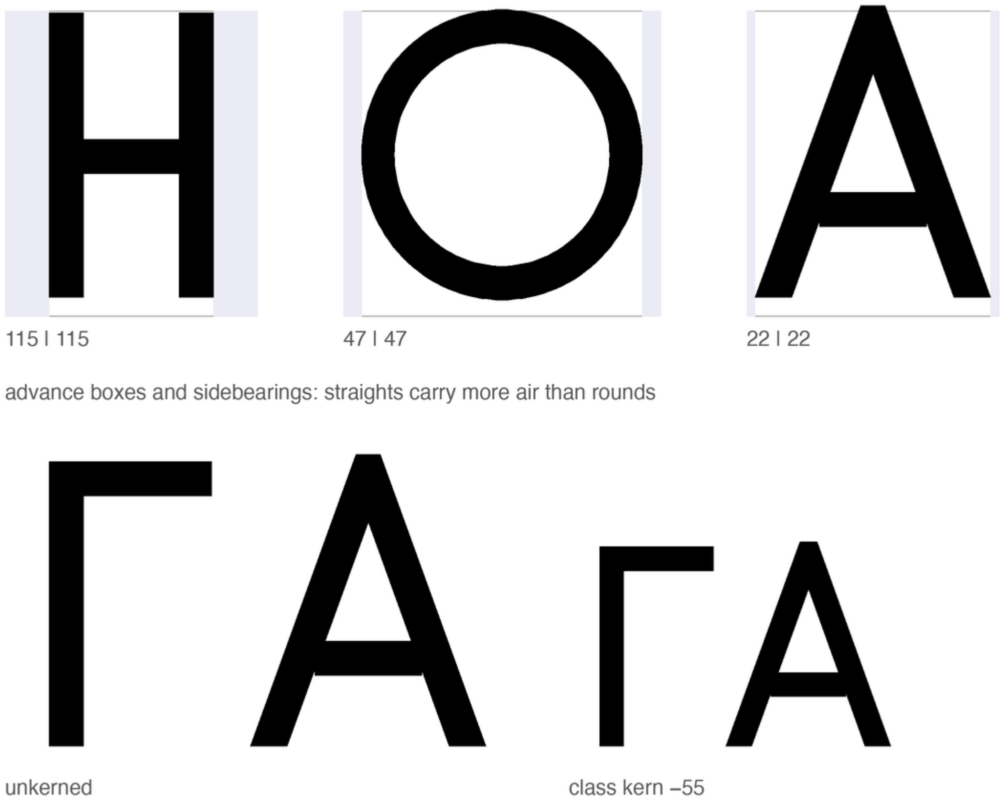

Figure 9: Fitting. Left: advance boxes and sidebearings for a straight, a round, and a diagonal letter — the classical hierarchy of white. Right: ΓA at raw advances and under the -55 class kern.

Second, *mark law*: every combining mark carries computed GPOS anchors — the macron of ā attaches 251 units lower than that of Ā; the retroflex hook of ʈ lands on the stem, not the visual center — so that the spec's statement "the hook attaches to the working stem" is not a description of intent but a checkable property of the artifact (Figure 14). Third, *verification culture*: the build is proofed by shaping with HarfBuzz and rendering through FreeType, the same code paths as deployment, after the even-odd episode of §4.2 taught us that a proof renderer can lie in both directions.

One design decision deserves its failure story told straight. The ejective digraphs (тӀ кӀ цӀ) were first fused with a full-height palochka, on the reasonable ground that the palochka *is* full-height. The result read as п. The shipping form raises the palochka to the upper half — echoing the apostrophe of scholarly romanization — and the standalone letter keeps its full height. The lesson generalizes: in ligature design, the components' identities matter less than the fusion's silhouette, because the reader receives only the silhouette (Figure 15).

8 The pipeline, precisely

Reproducibility demands specifics, so: the builder is ∼700 lines of Python on `fontTools`. Glyphs are declared as the centerline primitives of §2; expansion emits *dense polygon* outlines — circles at 48–128 points by radius, arcs at one point per 2.5° — with no Bézier segments and no boolean union: overlapping strokes are left overlapping, and nonzero winding resolves them at rasterization, with contour orientation fixed by signed area (solids positive, holes reversed). This choice assumes nonzero-winding fill throughout the consuming pipeline; export paths that convert outlines to even-odd fill or perform destructive path simplification require testing or pre-flattening — the concave-fold episode of §4.2 is exactly what such a consumer exposes. Both formats are cut from identical contours —

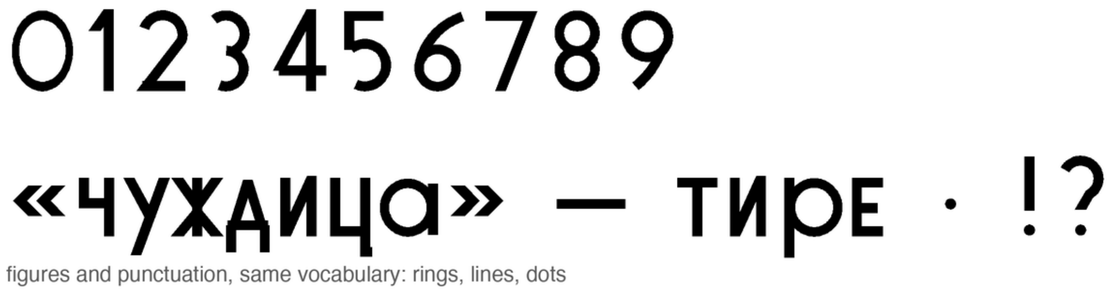

Figure 10: Figures and punctuation from the same four primitives. Tabular-ish digits, guillemets, dashes, the interpunct that doubles as the specimen's separator.

Figure 11: Case pairs. а б р ф are constructed, not scaled — bowl-and-stem, ascender, descender — while the remaining lowercase derives from the capital skeletons with descenders kept at full depth.

TrueType via `TTGlyphPen`, CFF via `T2CharStringPen` — and the four styles are not interpolated but re-evaluated: the same skeletons under different $(w,$ slant$)$, which makes one kind of style consistency — structural agreement across styles — a consequence of the source model rather than a per-glyph discipline; optical adequacy per style still had to be earned (§4.4, §5). The character map is generated from a single codepoint table including the NFC closure; GSUB and GPOS are emitted as AFDKO feature source and compiled by `feaLib`; the test harness shapes with `uharfbuzz` and rasterizes with FreeType — the deployment code paths, after a proof-renderer incident (§4.2) taught us not to trust any other.

The polygon decision deserves its numbers, because "dense polygons are harmless" is a claim, not a fact. Harmless here is measured: each style ships at 52–59 KB (TTF) and 52–54 KB (CFF) for 250+ glyphs — comparable to or smaller than mainstream hinted text fonts — because monoline outlines are few contours with cheap points; rasterizers flatten Béziers to line segments before scan conversion in any case; the face carries no hinting for the polygons to complicate; and PDF embedding is exercised by the document in hand. The genuine costs are two: no analytically exact extrema (irrelevant unhinted, but real), and larger source-to-outline conversion cost if a Bézier workflow is ever wanted — quadratic conversion via `cu2qu`-style fitting is the natural future path, and a variable-font export (weight and slant are already parameters) the natural successor.

9 At reading sizes

First, the intended use, stated narrowly: Chuzhditsa is designed for *embedded* settings — loanwords and names inside ordinary Cyrillic text, dictionary pronunciation fields, captions, pedagogy, and display — not for long-form running text, where its monoline geometry would read as display type does. The evaluation below targets that use: the face must keep its extension letters distinct, and their marks alive, at caption sizes inside someone else's paragraph.

A display-tested face can still die at nine points, and a parametric monoline has two structural advantages and one liability there. The advantages: no hairlines to drop out (the monoline's single weight is its own floor), and terminals cut flat rather than rounded, which antialiasing renders as clean

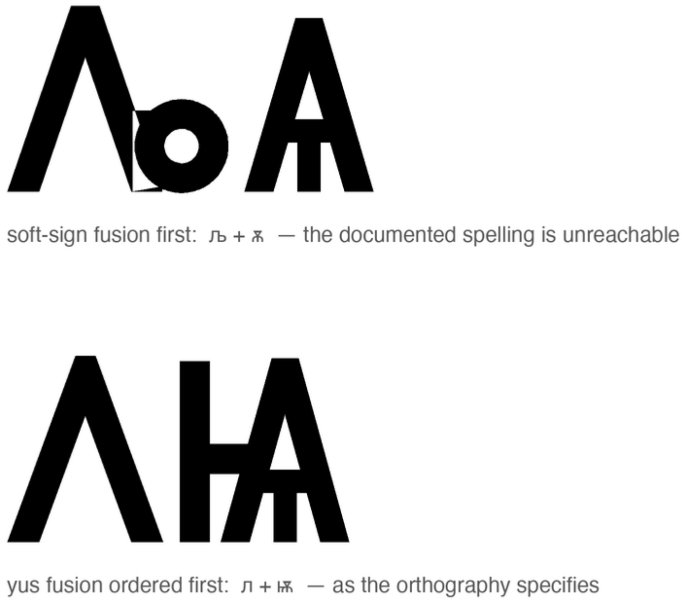

Figure 12: Lookup order as orthographic law. Above: soft-sign fusion first — л+ь fuse, stranding the big yus; the spelling льѫ cannot be typed. Below: the shipped order — yus fusion first.

steps rather than gray smears. The liability: no hinting — the fonts carry no instructions, and small-size fidelity is delegated wholly to the rasterizer's antialiasing, which is the modern default assumption but a stated one. Figure 17 is the honest test: the same sentence from display size down to footnote size, rendered through FreeType exactly as a reader's device would. The extension letters are the letters to watch — a breve, a hook, or an ogonek that survives at 13 pixels is the difference between a writing system and a specimen; this is where the mark-size and clamp decisions of the preceding sections either pay rent or default. Figure 16 puts the confusable pairs — native letter against extension letter — in front of the eye at three sizes, with the small sizes magnified pixel-for-pixel rather than smoothed. Beyond the specimen, Table 2 instruments the question: glyph pairs rendered through FreeType at fixed pixel sizes, ink thresholded at 50%, and measured for mark survival (ink pixels strictly above the base letter's topmost row) and confusable-pair contrast (symmetric difference of ink). The honest findings: every mark and every tested contrast is robust from 12 px upward; at 10 px the macron survives by a single pixel row and з/ȝ approach merger (2 px) — 12 px is this face's floor for reliable extension-letter contrast, and the counters of o remain open at every tested size in both weights.

px	breve ў	macron Ӣ	umlaut Ä	c/ç	з/ȝ	к/ķ	ᴇ/є	o counter R/B
10	2	1	3	9	2	3	9	open/open
12	2	3	4	16	9	9	16	open/open
14	4	3	5	22	4	6	17	open/open
18	7	8	6	18	17	15	31	open/open
24	10	12	10	42	12	22	54	open/open

Table 2: Instrumented small-size check, FreeType rendering, 50% ink threshold. Mark columns: ink pixels above the base letter's top. Pair columns: symmetric difference of ink in pixels. The 10 px row records the face's honest floor. The metric is comparative, not psychophysical: values below roughly 3–4 pixels are treated as suspect under visual inspection, and values are not expected to rise monotonically with size, since rasterization and thresholding interact with grid phase.

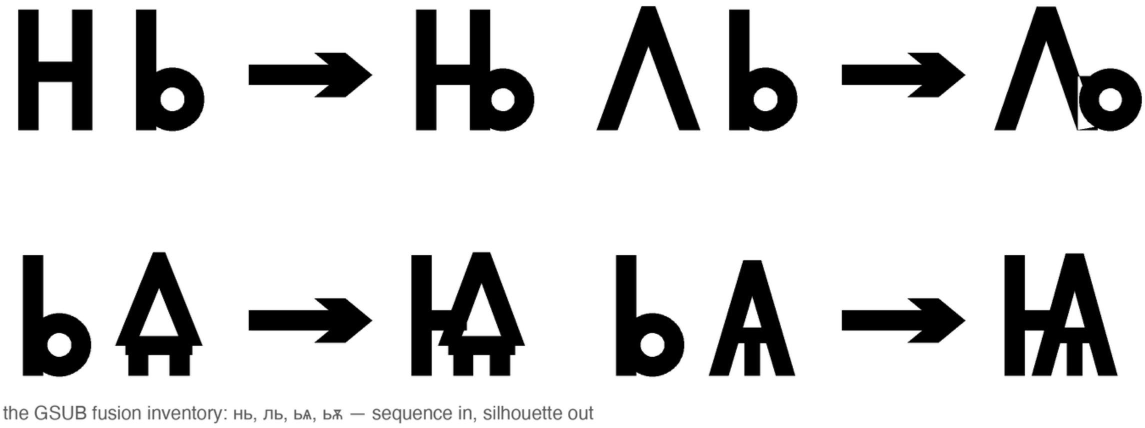

Figure 13: The GSUB fusion inventory: нь and ль into their Vuk-style fused forms; soft sign plus nasal into the medieval iotated yuses. Sequence in, silhouette out; the components' separate identities do not survive, by design.

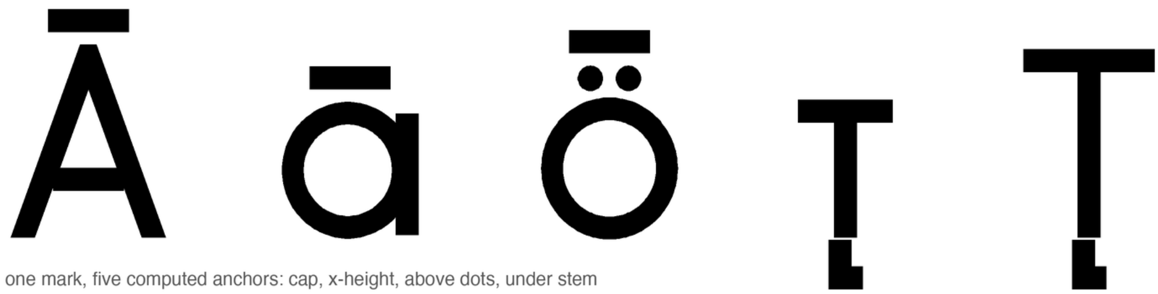

Figure 14: One mark, many computed homes. The macron seats at cap height on A, at x-height on a, above the baked-in dots of ö; the retroflex hook finds the stem of т in either case. No anchor was placed by hand.

10 Limitations

Dense polygon outlines trade analytic exactness for simplicity; the measured costs are small (§8) but a Bézier or variable-font export remains unbuilt. There is no optical-size axis, no language-specific localized forms (Bulgarian-form alternates would be the natural first set), and no mark-to-mark positioning, which caps stacked prosody. The register's visual layer is fragile under font fallback (§1's encoding distinction), behavior in office suites and mobile rendering stacks is untested, and — the largest open item — no reading study with Cyrillic readers has been run: every legibility claim here is instrumented or proofed, none is behavioral. The face is monoline by conviction but also by scope: the engine has no contrast axis, and adding one would promote it from seven hundred lines to a research project — precisely the slope on which METAFONT's practical reputation perished (Southall 1985). Kerning is class-based and shallow, and the parametric source makes a variable-font export the natural successor. The italic is a corrected oblique, not a cursive: a true italic would need new skeletons, which is to say, new writing.

11 Conclusion

A typeface for a designed writing system was built as the writing system was designed: by rules, argued in the open, verified by machines, corrected by the eye. The construction vindicates a modest form of Knuth's programme — parameters within one design language, never across all of them — and confirms an old suspicion of the drawing office: that most of what reads as beauty in geometric type is

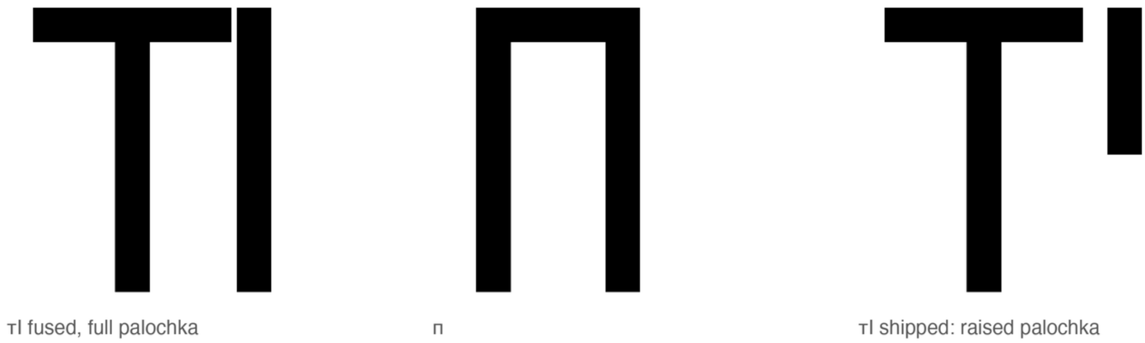

Figure 15: The ejective ligature's failure and repair. Left: тӏ fused with a full-height palochka. Center: п. The resemblance is disqualifying. Right: the shipped fusion — the palochka rises to the upper half, echoing the apostrophe of scholarly romanization.

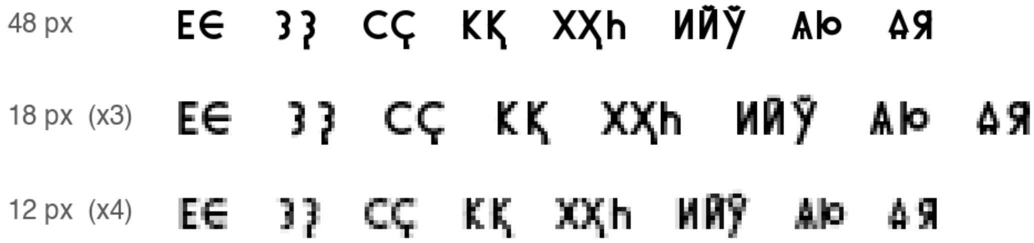

Figure 16: Confusable pairs at reading sizes, FreeType rendering: е/є, з/ӡ, с/ҫ, к/қ, х/ҳ/һ, и/й/ў, а/ю, а/я. The 18 and 12 pixel rows are nearest-neighbor magnifications of the actual rasters — what the renderer produced, not what the vector promises.

a ledger of small, principled lies to Euclid, and that the ledger, at last, can be source code. The reusable contribution is not the Chuzhditsa aesthetic: it is the demonstration that a small stroke grammar, an explicit optical canon, and a shaping-test harness suffice to produce and verify a coherent multi-style script-extension family — a recipe portable to any register, any script, any design language willing to state its rules. The face itself closes the argument (Figure 18).

References

[1] Bigelow, C. & K. Holmes. 1993. The design of a Unicode font. *Electronic Publishing* 6(3). 289–305.

[2] Bringhurst, R. 2004. *The Elements of Typographic Style*, 3rd edn. Point Roberts, WA: Hartley & Marks.

[3] Burke, C. 1998. *Paul Renner: The Art of Typography*. London: Hyphen Press.

[4] Dwiggins, W. A. 1937. Memoranda to C. H. Griffith, 3 July 1937. Chauncey Hawley Griffith Papers, University of Kentucky Libraries.

[5] Dwiggins, W. A. 1940. *WAD to RR: A Letter about Designing Type*. Cambridge, MA: Harvard College Library.

[6] Hershey, A. V. 1967. *Calligraphy for Computers*. Report NWL-2101. Dahlgren, VA: U.S. Naval Weapons Laboratory.

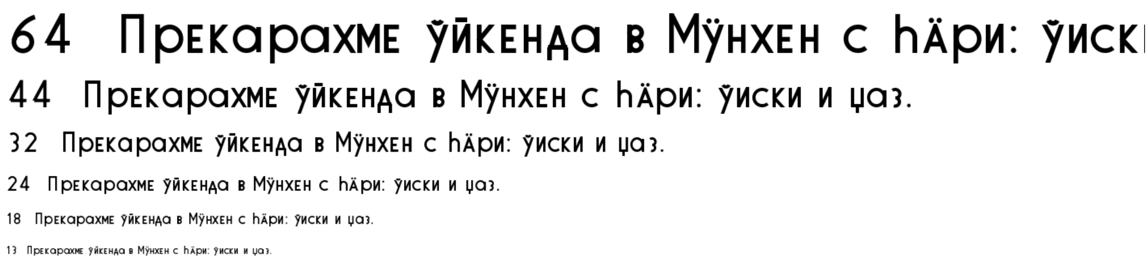

Figure 17: Waterfall, FreeType rendering, 64 down to 13 pixels. The marks — breve, macron, umlaut, hooks — are the stress test: a citation register that loses its diacritics at caption sizes has lost its phonology.

[7] Hofstadter, D. R. 1982. Metafont, metamathematics, and metaphysics: Comments on Donald Knuth's article "The concept of a meta-font". *Visible Language* 16(4). 309–338.

[8] Johnston, E. 1906. *Writing & Illuminating, & Lettering*. London: John Hogg.

[9] Karow, P. 1994. *Font Technology: Methods and Tools*. Berlin: Springer.

[10] Knuth, D. E. 1982. The concept of a meta-font. *Visible Language* 16(1). 3–27.

[11] Knuth, D. E. 1986. *The METAFONTbook*. Reading, MA: Addison-Wesley.

[12] Legros, L. A. & J. C. Grant. 1916. *Typographical Printing-Surfaces: The Technology and Mechanism of Their Production*. London: Longmans, Green.

[13] Noordzij, G. 2005. *The Stroke: Theory of Writing*. London: Hyphen Press.

[14] Smeijers, F. 1996. *Counterpunch: Making Type in the Sixteenth Century, Designing Typefaces Now*. London: Hyphen Press.

[15] Southall, R. 1985. *Designing New Typefaces with Metafont*. Report STAN-CS-85-1074. Stanford, CA: Stanford University, Dept. of Computer Science.

[16] Tinker, M. A. 1963. *Legibility of Print*. Ames, IA: Iowa State University Press.

[17] Tracy, W. 1986. *Letters of Credit: A View of Type Design*. London: Gordon Fraser.

[18] Йончев, В. & О. Йончева. 1982. *Древен и съвременен български шрифт*. София: Български художник.

[19] Zhukov, M. 1996. The peculiarities of Cyrillic letterforms: Design variation and correlation in Russian typefaces. *Typography Papers* 1.

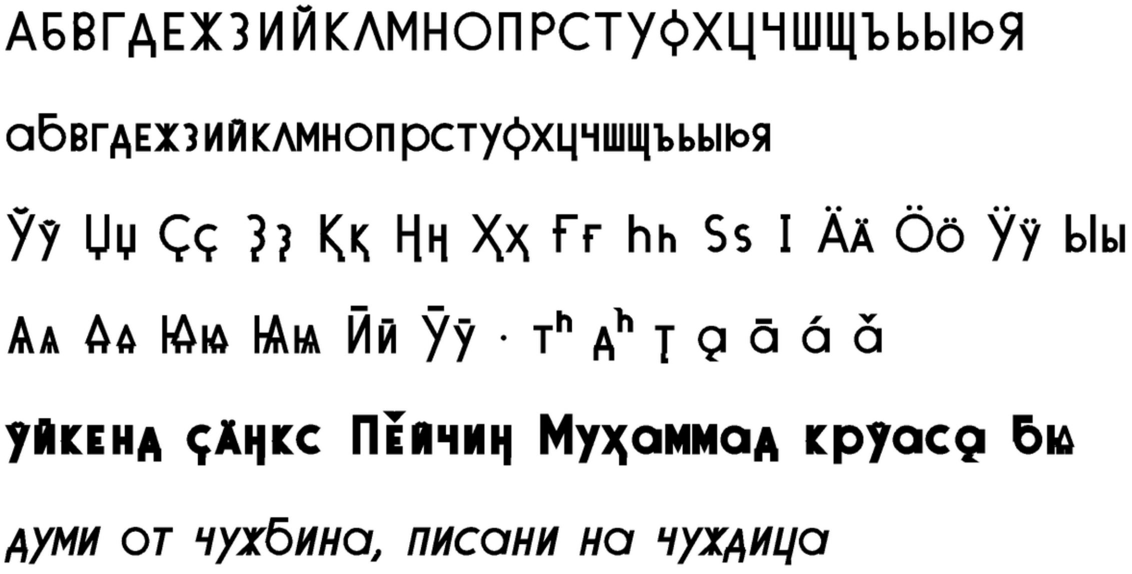

Figure 18: Chuzhditsa v2.5: the Slavic base, the designed extension inventory, and the four styles — every glyph, joint, terminal, and correction in this specimen is the output of the seven hundred lines this paper described.